

IPO

Interacción Persona Ordenador

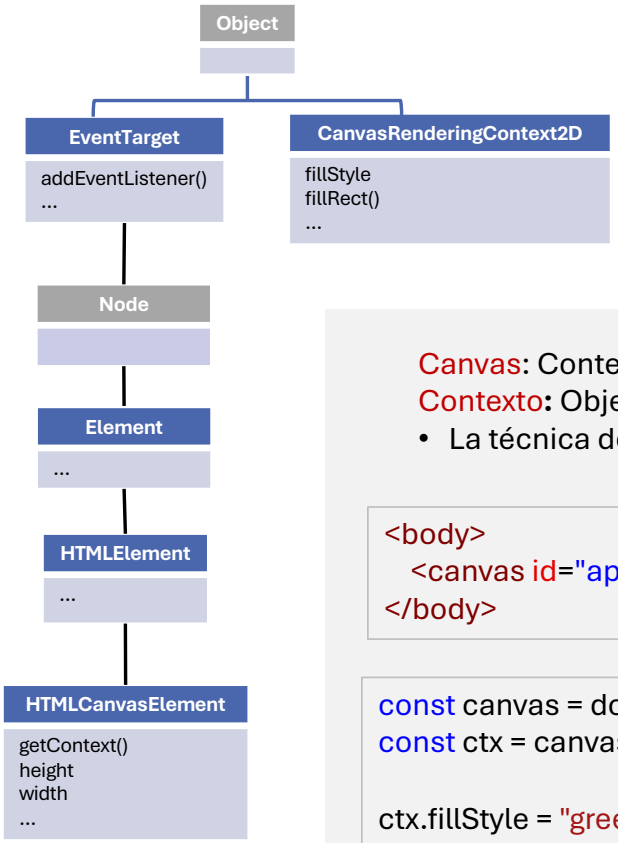
E.T.S. Ingeniería Informática
Universidad de Sevilla

Canvas API



1. Introducción
2. Rectángulos
3. Caminos
4. Interacción
5. Animación

Introducción



Canvas API proporciona interfaces que permiten

- visualización de datos y figuras
- animaciones y juegos gráficos
- procesamiento en tiempo real de videos
- edición de imágenes

Canvas: Contenedor HTML sobre el que se visualizaran los resultados: actúa como un lienzo en blanco
Contexto: Objeto vinculado al elemento canvas que posibilita la elaboración y visualización del dibujo

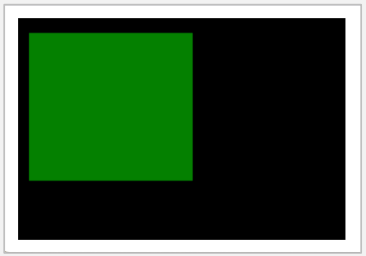
- La técnica de dibujo es secuencial (el orden influye en el resultado) y la visualización es inmediata

```
<body>
  <canvas id="appCanvas" class="app-canvas"></canvas>
</body>
```

```
const canvas = document.getElementById("appCanvas");
const ctx = canvas.getContext("2d");

ctx.fillStyle = "green";
ctx.fillRect(10, 10, 150, 100);
```

El sistema de coordenadas del lienzo usa enteros positivos siendo la esquina superior izquierda el punto (0,0)



CanvasRenderingContext2D Interfaz encargada del contexto **2d** especializada en realizar dibujos bidimensionales

Rectángulos

La primitiva básica de dibujo en el contexto 2d es el rectángulo

fillRect(x, y, w, h)	Dibuja el rectángulo completo
strokeRect(x, y, w, h)	Dibuja sólo el contorno del rectángulo
clearRect(x, y, w, h)	Borra el espacio ocupado por el rectángulo

Argumentos
- x, y: Posición de la esquina superior izquierda del rectángulo
- w, h: Dimensión del rectángulo (anchura y altura)

CanvasRenderingContext2D

fillRect()
strokeRect()
clearRect()
...
fillStyle
strokeStyle
lineWidth
shadowColor
shadowBlur
shadowOffesetX
shadowOfsetY
lineJoin
...

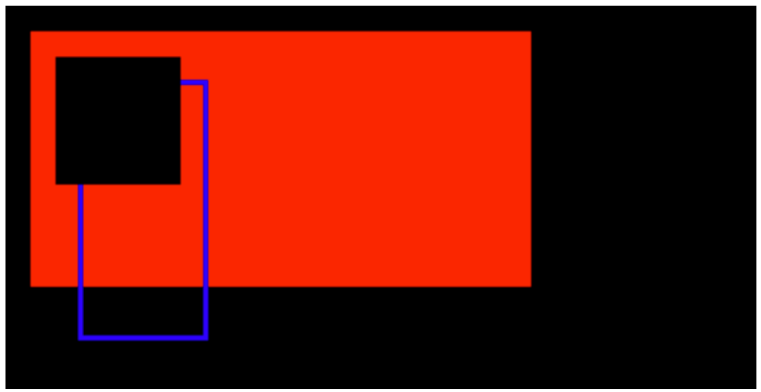
La interfaz ofrece propiedades para estilizar los dibujos

Los estilos se aplican y mantienen según el orden de aparición

```
ctx.fillStyle = 'red';
ctx.fillRect(10, 10, 200, 100);

ctx.strokeStyle = 'blue';
ctx.lineWidth = 2;
ctx.strokeRect(30, 30, 50, 100);

ctx.clearRect(20, 20, 50, 50);
```



La notación de los colores es equivalente a la proporcionada por CSS y los valores para de dimensiones son en pixeles

Caminos: Introducción

Camino o ruta (path): Secuencia de puntos que conforma un trazado

Los caminos son la primitiva que permiten dibujar en el contexto 2d figuras personalizadas

CanvasRenderingContext2D

fillRect()
strokeRect()
clearRect()
...
fillStyle
strokeStyle
lineWidth
shadowColor
shadowBlur
shadowOffesetX
shadowOfsetY
lineJoin
...
beginPath()
lineTo()
moveTo()
closePath()
stroke()
fill()
...

beginPath()	Comienzo de un nuevo camino. Los puntos son añadidos según su aparición
closePath()	Conecta el punto inicial y final del camino. OJO: Los caminos pueden ser abiertos o cerrados
lineTo(x,y)	Añade (x,y) al camino activo registrando que existe una línea desde la posición anterior hasta (x,y)
moveTo(x, y)	Añade (x,y) al camino activo sin conexión con el anterior. Simula el efecto de levantar el lápiz sin dibujar
stroke()	Dibuja los trazos del camino activo siguiendo el estilo strokeStyle
fill()	Dibuja el interior de la forma del camino aplicando el estilo fillStyle

```
ctx.beginPath();
ctx.moveTo(10, 10);
ctx.lineTo(30, 50);
ctx.lineTo(40, 30);
ctx.closePath();
ctx.strokeStyle = "red";
ctx.lineWidth = 4;
ctx.stroke();

ctx.beginPath();
ctx.moveTo(60, 10);
ctx.lineTo(80, 50);
ctx.lineTo(90, 30);
ctx.fillStyle = "orange";
ctx.fill();
```



- Solo hay un camino activo en cada instante de tiempo
- Las rutas no se visualizan hasta que se ejecutan stroke() o fill()
- Los estilos afectan de forma global a todo el camino

Caminos: Trazados curvos

Los caminos engloban trazos tanto rectos como curvos

CanvasRenderingContext2D

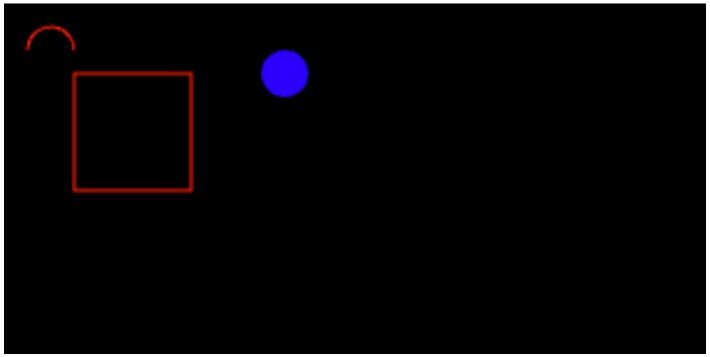
fillRect()
strokeRect()
clearRect()
...
fillStyle
strokeStyle
lineWidth
shadowColor
shadowBlur
shadowOffesetX
shadowOfsetY
lineJoin
...
beginPath()
lineTo()
moveTo()
closePath()
stroke()
fill()
...
arc()
rect()
arcTo()
ellipse()
bezierCurveTo()
cuadraticCurveTo()
...

arc(x, y, r, inicio, fin)

Arco de radio r con centro en x, y que comienza y acaba en los ángulos (en radianes) inicio y fin

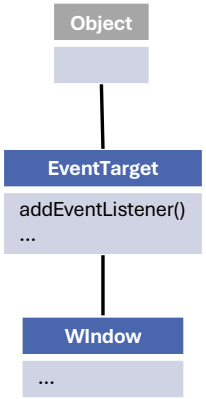
```
ctx.beginPath();
ctx.arc(20, 20, 10, Math.PI, 2 * Math.PI);
ctx.rect(30, 30, 50, 50);
ctx.strokeStyle = ”red";
ctx.stroke();

ctx.beginPath();
ctx.moveTo(100, 30);
ctx.arc(120, 30, 10, 0, 2 * Math.PI);
ctx.fillStyle = "blue";
ctx.fill();
```



- El método rect(x,y,w,h) permite incluir rectángulos dentro de un camino
- Existen otros métodos para realizar trazos curvos:
  arcTo() ellipse() bezierCurveTo() cuadraticCurveTo()

Interacción: Gestión de eventos



Las figuras dibujadas en el elemento canvas **NO** tienen reflejo en el árbol DOM
Es factible modificar el contenido del canvas reaccionando a eventos desencadenados en

- elementos del DOM (por ejemplo, el propio canvas)
- la ventana (window)

La ventana se encarga de atender a la pulsación de las teclas Cursor Arriba y Abajo

Para evitar la superposición de imágenes es habitual borrar el contenido antes de efectuar un cambio

```
import { Cuadrado } from "./Cuadrado.js";

const canvas = document.getElementById("appCanvas");
const ctx = canvas.getContext("2d");

const cuadro= new Cuadrado(canvas, 50, 50, 50, 'blue', 2);
cuadro.dibuja();

window.addEventListener('keydown', mueveCuadrado)

function mueveCuadrado(event) {
    const key = event.key;
    switch (key) {
        case 'ArrowUp':
            cuadro.sube()
            break
        case 'ArrowDown':
            cuadro.baja()
            break;
        default:
            return;
    }

    ctx.clearRect(0, 0, canvas.width, canvas.height);
    cuadro.dibuja();
}
```

```
export class Cuadrado {

    constructor(canvas,x,y,lado,color,dy) {
        this.canvas = canvas;
        this.lado = lado;
        this.x = x;
        this.y = y;
        this.color = color;
        this.dy = dy;
        this.ctx = this.canvas.getContext("2d");
    }

    dibuja() {
        this.ctx.fillStyle = this.color;
        this.ctx.fillRect(this.x, this.y, this.lado, this.lado);
    }

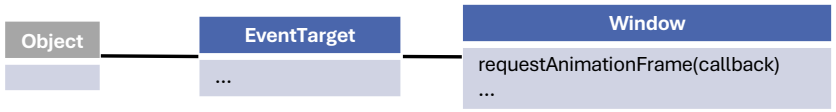
    sube() {
        if (this.y - this.dy >= 0)  this.y -= this.dy;
    }

    baja() {
        if (this.y + this.dy + this.lado <= this.canvas.height)   this.y += this.dy;
    }
}
```

La clase define el objeto que se verá afectado por la interacción

Animación: requestAnimationFrame

Las animaciones se consiguen mediante la implementación de un ciclo limpiar-pintar



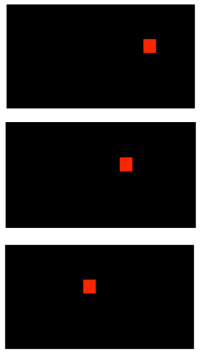
requestAnimationFrame(callback)

Solicita al navegador que la función *callback* comience su ejecución en el siguiente ciclo de refresco del monitor

- sincroniza el ciclo de refresco con el bucle de animación
- se alcanza mayor precisión que las interfaces Interval o Timeout

La tasa de refresco del monitor suele ser 60fps

La animación consiste en un cuadrado que se desplaza en el eje X, de forma que si alcanza los bordes del lienzo cambia el sentido del movimiento



Se recomienda que el ciclo de animación comience solicitando la sincronización con el próximo ciclo de refresco

```
import { Cuadrado } from "./Cuadrado.js";

const canvas = document.getElementById("appCanvas");
const ctx = canvas.getContext("2d");

const cuadrado = new Cuadrado(canvas, 20, 'red', 50, 50, 2);

function animaCuadrado() {
   requestAnimationFrame(animaCuadrado);

   ctx.clearRect(0, 0, canvas.width, canvas.height);
   cuadrado.dibuja();
   cuadrado.actualiza();
}

requestAnimationFrame(animaCuadrado);
```

```
export class Cuadrado {

  constructor(canvas, lado, color, x, y, dx) {
    this.canvas = canvas;
    this.ctx = canvas.getContext("2d");
    this.lado = lado;
    this.color = color;
    this.x = x;
    this.y = y;
    this.dx =dx;
  }

  dibuja() {
     this.ctx.fillStyle = this.color;
     this.ctx.fillRect(this.x, this.y, this.lado, this.lado);
  }

  actualiza() {
     // Colisiones: Cambia el sentido al alcanzar los bordes
     if (this.x + this.dx + this.lado > this.canvas.width
           || this.x + this.dx < 0) {
        this.dx = -this.dx;
     }
     this.x += this.dx;
  }
}
```

dx refleja la velocidad, i.e. la cantidad que se avanza en el eje X cada vez que hay una actualización


UNIVERSIDAD DE SEVILLA
UNIVERSIDAD Ð SEVILLA